



Advanced Time Series Analysis and Forecasting

Chapter 12: Machine learning and deep learning for time series



Daniel Traian PELE

Bucharest University of Economic Studies

IDA Institute Digital Assets

Blockchain Research Center

AI4EFin Artificial Intelligence for Energy Finance

Romanian Academy, Institute for Economic Forecasting

MSCA Digital Finance

Acronyms used in this material (1/3)

- ▣ ACF: Autocorrelation Function
- ▣ AI: Artificial Intelligence
- ▣ AR: AutoRegressive (model); in event studies: Abnormal Return
- ▣ ARMA: AutoRegressive Moving Average (model)
- ▣ ARX: AutoRegressive model with eXogenous variables
- ▣ BET: Bucharest Exchange Trading (index)
- ▣ BPTT: BackPropagation Through Time
- ▣ CPU: Central Processing Unit
- ▣ CRPS: Continuous Ranked Probability Score
- ▣ CV: Cross-Validation
- ▣ DGP: Data-Generating Process
- ▣ DM: Diebold–Mariano (test)
- ▣ ENTSO-E: European Network of Transmission System Operators for Electricity
- ▣ ERM: Empirical Risk Minimisation
- ▣ ES-RNN: Exponential Smoothing – Recurrent Neural Network (hybrid)
- ▣ ETS: Error, Trend, Seasonality (exponential smoothing state space models)
- ▣ ETT: Electricity Transformer Temperature (data set)
- ▣ EU: European Union
- ▣ FFT: Fast Fourier Transform
- ▣ GARCH: Generalised AutoRegressive Conditional Heteroskedasticity
- ▣ GRN: Gated Residual Network
- ▣ GRU: Gated Recurrent Unit
- ▣ HAC: Heteroskedasticity and Autocorrelation Consistent (standard errors)
- ▣ HAR: Heterogeneous AutoRegressive (model)
- ▣ HGB: Histogram Gradient Boosting
- ▣ HICP: Harmonised Index of Consumer Prices

Acronyms used in this material (2/3)

- ▣ HLN: Harvey–Leybourne–Newbold (small-sample correction of the DM test)
- ▣ KKT: Karush–Kuhn–Tucker (conditions)
- ▣ LLM: Large Language Model
- ▣ LOOCV: Leave-One-Out Cross-Validation
- ▣ LSTM: Long Short-Term Memory (network)
- ▣ LTSF: Long-Term Series Forecasting
- ▣ MA: Moving Average
- ▣ MAE: Mean Absolute Error
- ▣ MAPAE: Mean Absolute Predictive Accuracy Error
- ▣ MAPE: Mean Absolute Percentage Error
- ▣ MASE: Mean Absolute Scaled Error
- ▣ MCS: Model Confidence Set
- ▣ MIMO: Multiple-Input Multiple-Output (forecast strategy)
- ▣ ML: Machine Learning
- ▣ MLP: MultiLayer Perceptron
- ▣ MPAE: Mean Predictive Accuracy Error
- ▣ MSE: Mean Squared Error
- ▣ MT: Malta (country code)
- ▣ NL: NonLinear (shrinkage)
- ▣ OLS: Ordinary Least Squares
- ▣ OOS: Out-Of-Sample
- ▣ OWA: Overall Weighted Average (of sMAPE and MASE relative to Naive 2)
- ▣ QLIKE: Quasi-LIKElihood loss
- ▣ RMSE: Root Mean Squared Error
- ▣ RNN: Recurrent Neural Network
- ▣ RV: Realised Variance

Acronyms used in this material (3/3)

- ▣ **SGD: Stochastic Gradient Descent**
- ▣ **SHAP: SHapley Additive exPlanations**
- ▣ **SPA: Superior Predictive Ability (test)**
- ▣ **TCN: Temporal Convolutional Network**
- ▣ **TFT: Temporal Fusion Transformer**
- ▣ **TSA: Time Series Analysis**
- ▣ **US: United States**
- ▣ **VC: Vapnik–Chervonenkis (dimension)**
- ▣ **VIX: CBOE Volatility IndeX**
- ▣ **WIG20: Warszawski Indeks Giełdowy 20 (the Warsaw Stock Exchange index of the 20 largest companies)**

Today's question and route

- **Question:** when do flexible learners (penalised regressions, tree ensembles, deep networks) forecast time series better than strong statistical baselines, and how do we know it honestly?
 - ▶ dependence breaks the i.i.d. logic of machine learning: validation, generalisation and inference all change
- **Route** of the chapter
 - ▶ learning from dependent data: mixing, bounds, when cross-validation is valid
 - ▶ global and local models; lasso and its relatives in high dimension; tree ensembles in depth
 - ▶ recurrent and convolutional networks; attention and Transformers; N-BEATS, N-HiTS, DeepAR, TFT
 - ▶ interpretation and the honest benchmark: baselines, tests, data snooping
- We build on [TSA, Chapter 9](#) (lag features, walk-forward validation, trees, a small MLP, M4 and M5) and on [Chapter 1](#) (DM, MCS, scoring rules); Seminar 12 comes before this lecture

Learning outcomes

- ▣ State when K-fold cross-validation is valid for autoregressive prediction, and design blocked, purged and rolling-origin validation that avoids leakage
- ▣ Derive the lasso, adaptive lasso and elastic net solutions, state their consistency conditions for dependent data, and explain why naive inference after selection fails
- ▣ Explain the bias–variance mechanics of bagging, random forests, quantile regression forests and gradient boosting with constraints
- ▣ Derive vanishing gradients in recurrent networks and the role of gates; read the architectures of TCN, Transformers, N-BEATS, N-HiTS, DeepAR and TFT
- ▣ Run a pre-registered comparison against strong baselines with DM, MCS and seeds reported, and interpret Shapley values and attention correctly

Reading, data and tools

- ▣ Validation and learning theory: [Bergmeir, Hyndman and Koo \(2018\)](#); [Racine \(2000\)](#); [Mohri and Rostamizadeh \(2010\)](#); [Hewamalage, Ackermann and Bergmeir \(2023\)](#)
 - ▶ High dimension and trees: [Tibshirani \(1996\)](#); [Zou \(2006\)](#); [Basu and Michailidis \(2015\)](#); [Meinshausen \(2006\)](#); [Friedman \(2001\)](#); [Ke et al. \(2017\)](#)
 - ▶ Deep learning: [Hochreiter and Schmidhuber \(1997\)](#); [Vaswani et al. \(2017\)](#); [Zeng et al. \(2023\)](#); [Oreshkin et al. \(2020\)](#); [Salinas et al. \(2020\)](#); [Lim et al. \(2021\)](#); [survey Benidis et al. \(2022\)](#)
- ▣ Python Quantlets of this chapter: [Quantlets/Ch_12](#)
 - ▶ `numpy`, `scikit-learn` and `PyTorch` on a CPU: small networks, fixed seeds, runs of minutes
- ▣ Lecture notebook: [open in Google Colab](#)

Data used in this chapter

Series	Source, sample	Use
Romanian electricity load, hourly	Energy-Charts (ENTSO-E transparency data), 1 369 days to 30 September 2026	trees, Transformers, DLinear
HICP inflation, 27 EU countries	Eurostat prc_hicp_minr, annual rates, monthly, January 2000 – August 2026	global models, lasso
S&P 500 and five other indices, realised variance	Heber et al. (2009) : 5-minute RV, 5 552 days to 25 February 2022	deep models against HAR
M4 hourly subset	Makridakis, Spiliotis and Assimakopoulos (2020) : 414 series of 700–960 hours, horizon 48	N-BEATS, N-HiTS, DeepAR

- Every comparison has a forecast period fixed before the models are run; losses, tests and seeds are reported with it

Four generations of forecasting models



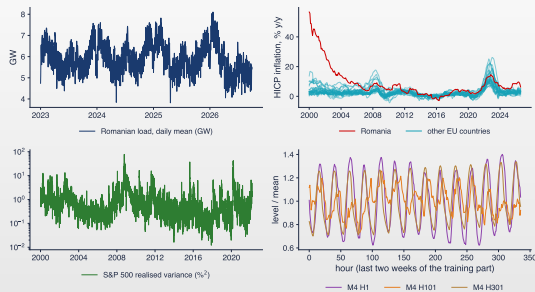
Geoffrey Hinton, Nobel Prize in Physics 2024

Photo: Arthur Petron (2024); CC BY-SA 4.0;

[Wikimedia Commons](#)

- **Statistical:** ARIMA, ETS, HAR, state space: few parameters, one model per series
- **Machine learning:** penalised regressions, forests, boosting on lag features, often global
- **Deep sequence models:** RNN, LSTM, TCN, Transformers, N-BEATS, DeepAR; backpropagation [Rumelhart, Hinton and Williams \(1986\)](#)
- **Foundation models:** pretrained on many series, used zero-shot ([Chapter 13](#))
- Each generation must beat the previous one out of sample, with tests, on the same data

Four data sets, four kinds of structure



- Romanian load (daily mean, 6.06 GW on average); EU inflation (Romania in red); S&P 500 realised variance (log scale); three M4 hourly series (last two weeks)

Interpreting the four data sets

- ▣ Load: strong daily and weekly seasonality, holidays, slow annual cycle: calendar structure that trees and linear models capture well
- ▣ Inflation: 27 short, co-moving series with a common shock in 2021–2023: the natural ground for global models and for the high-dimensional panel
- ▣ Realised variance: long memory and heavy tails; the HAR benchmark of [Chapter 8](#) is hard to beat
- ▣ M4 hourly: many related series of the same frequency, the setting where global deep models were first shown to win

Known from TSA, and new here

- Known (TSA, Chapter 9): lag features, walk-forward validation, leakage by example, ridge and lasso, trees, random forests, gradient boosting, a small MLP, conformal basics, M4 and M5
- New: the statistics behind the recipes
 - ▶ generalisation under mixing; when cross-validation is valid; global models
 - ▶ selection consistency and post-selection inference; quantile forests; constrained boosting
 - ▶ gradients through time; attention; basis expansion; likelihood-based deep models; interpretation
- Replications: Bergmeir, Hyndman and Koo (2018, Section 4); Montero-Manso and Hyndman (2021) on EU inflation; Zeng et al. (2023) protocol on Romanian load; the M4 hourly evaluation

Risk, empirical risk and the generalisation gap (1/2)

- A forecast rule f from a class $\mathcal{F}$ is judged by its expected loss under the stationary law

$$R(f) = \mathbb{E} \ell(Y_{t+h}, f(X_t)), \quad \hat{R}_n(f) = \frac{1}{n} \sum_{t=1}^n \ell(y_{t+h}, f(x_t))$$

- ▶ $R(f)$: **risk**; $\hat{R}_n(f)$: **empirical risk** on n training pairs; ℓ : loss (e.g. squared error); X_t : features known at t ; h : horizon

- Empirical risk minimisation (ERM) and the generalisation gap

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_n(f), \quad R(\hat{f}) - \hat{R}_n(\hat{f}) \leq \sup_{f \in \mathcal{F}} |R(f) - \hat{R}_n(f)|$$

- ▶ the gap: how much worse the fitted rule does on new data than on the training data

Risk, empirical risk and the generalisation gap (2/2)

- i.i.d. data: the uniform deviation shrinks with the complexity of the class

$$\sup_{\mathcal{F}} |R - \hat{R}_n| = O_p\left(\sqrt{\text{complexity}(\mathcal{F})/n}\right)$$

- ▶ complexity: VC dimension or Rademacher complexity, roughly the number of patterns the class can fit
- Dependent data: the n observations carry less information; the bound holds with an **effective sample size** smaller than n
 - ▶ and only if the series forgets its past fast enough: mixing
- Non-stationarity (breaks, [Chapter 2](#)) adds a second gap: the future law differs from the training law; no bound covers it

Mixing coefficients

- β -mixing: how far the distant future is from being independent of the past

$$\beta(a) = \sup_t \mathbb{E} \sup_{B \in \sigma(X_{t+a}, \dots)} |P(B \mid \sigma(\dots, X_t)) - P(B)|$$

- ▶ $\sigma(\dots, X_t)$: the information in the past up to t ; $\sigma(X_{t+a}, \dots)$: events B of the future beyond the gap a ;
 α -mixing uses $|P(A \cap B) - P(A)P(B)|$ instead
- ▶ mixing: $\beta(a) \rightarrow 0$ as the gap $a \rightarrow \infty$: the far future is almost independent of the past
- Stationary ARMA with continuous innovations, GARCH and Markov-switching models with ergodic chains are **geometrically** β -mixing: $\beta(a) \leq Cr^a$, $C > 0$, $0 < r < 1$
 - ▶ long memory ([Chapter 10](#)) is not geometrically mixing: the effective sample size shrinks much more
- Mixing is assumed, rarely tested: it is a statement about the data-generating process, not about the sample

Blocking and a generalisation bound

- Yu (1994): split the sample into 2μ alternating blocks of length a ; under β -mixing the odd blocks behave like μ **independent** blocks up to an error $\mu\beta(a)$
 - ▶ i.i.d. tools then apply with $\mu = n/(2a)$ observations instead of n

- Mohri and Rostamizadeh (2010): for a uniformly stable learner, with probability at least $1 - \delta$

$$R(\hat{f}) \leq \hat{R}_n(\hat{f}) + O(\kappa_n a) + O\left(\sqrt{\log(1/\delta')/\mu}\right), \quad \delta' = \delta - 2(\mu - 1)\beta(a)$$

- ▶ κ_n : uniform stability, the largest change of the loss when one training point changes; δ : confidence level
 - ▶ trade-off in a : long blocks make $\beta(a)$ small but leave few blocks μ ; ridge and regularised models are uniformly stable, deep networks trained by SGD only approximately
- The same blocking idea gives blocked bootstrap (Chapter 0) and blocked cross-validation

Cross-validation for time series: the schemes

- ▣ **Random K-fold**: rows of the lag matrix $(y_t; y_{t-1}, \dots, y_{t-p})$ assigned to folds at random
- ▣ **Non-dependent CV** Bergmeir, Hyndman and Koo (2018): the same folds, but training rows within p lags of a test row are removed
- ▣ **Blocked K-fold** and **hv-block** Burman, Chow and Nolan (1994), Racine (2000): contiguous test blocks, a gap of h rows (purge) on each side
- ▣ **Out-of-sample (OOS)** and **rolling origin**: test only on the end; the only scheme that mimics real use, at the price of a high variance
- ▣ The question is not which scheme is “correct” but which estimates the future risk with small bias and small variance

When K-fold is valid: Bergmeir, Hyndman and Koo (2018)

- A nonlinear autoregression with **uncorrelated** errors Bergmeir, Hyndman and Koo (2018)

$$y_t = g(y_{t-1}, \dots, y_{t-p}; \theta) + \varepsilon_t$$

- ▶ g : any regression function (linear, tree, network) with parameters θ ; p : number of lags; ε_t : errors, $\text{Cov}(\varepsilon_t, \varepsilon_s) = 0$ for $t \neq s$
- ▶ the rows of the lag matrix are then exchangeable enough: a test row shares no error with a training row
- ▶ result: the K-fold estimate of the MSE is asymptotically unbiased, and has a lower variance than OOS because every row is used
- It fails when the residuals are autocorrelated: an under-specified model, overlapping h -step targets, or features that identify time
 - ▶ then a test error is predictable from the neighbouring training errors: optimistic estimates
- Practical rule: check the residual ACF of the fitted model (Ljung–Box, TSA, Chapter 2) before trusting random K-fold

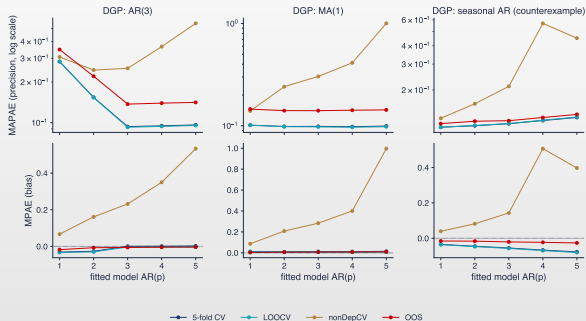
Replication design: Bergmeir, Hyndman and Koo (2018, Section 4)

- ▣ Series of length 200: in-set 140, out-set 60; models AR(1)–AR(5) by OLS; one-step forecasts
- ▣ Procedures on the in-set: 5-fold CV, LOOCV, non-dependent CV ($p = 5$), OOS (last 20%); “true” error: the model on the whole in-set, RMSE on the out-set
- ▣ 1 000 trials per DGP: stationary AR(3) with random coefficients, invertible MA(1), and a seasonal AR fitted to US accidental deaths 1973–1978 ($\phi = 0.76$, $\Phi_{12} = 0.85$) as the counterexample
- ▣ Precision and bias of each procedure

$$\text{MAPAE} = \text{mean} |\widehat{PE} - PE|, \quad \text{MPAE} = \text{mean} (\widehat{PE} - PE)$$

- PE : the true prediction error (RMSE on the out-set); $\widehat{PE}$: its estimate by the procedure; averages over the trials; $\text{MPAE} < 0$: optimistic

Which validation estimates the future error?



□ Top: precision (log scale); bottom: bias, by fitted AR order and data-generating process

Interpreting the replication

- ▣ AR(3) data, AR(3) model: 5-fold CV has MAPAE 0.094 against 0.137 for OOS, bias 0.001: as in the paper, CV is unbiased and more precise
- ▣ Under-specified AR(1) on AR(3) data: residuals are autocorrelated and CV loses its edge (MAPAE 0.284 against 0.349)
- ▣ Seasonal counterexample: every $\text{AR}(p \leq 5)$ misses lag 12; CV is biased down (MAPAE -0.077 for AR(5)) and no more precise than OOS (0.130 against 0.136)
- ▣ Non-dependent CV removes so many rows that its models are poor: bias 0.232 for AR(3); the price of independence is paid in data

Leakage through overlapping targets and time features

- Target $z_t = \frac{1}{h} \sum_{j=1}^h y_{t+j}$ (a month of volatility, a quarter of sales): $\text{corr}(z_t, z_{t+1}) \geq (h-1)/h$ when y is white noise, more when y is persistent
 - ▶ a random fold puts z_{t-1} and z_{t+1} in training when z_t is tested: the answer is almost in the training set
- A time index or a date feature lets a flexible learner **locate** the test row between its neighbours
- Simulation: AR(1) with $\phi = 0.9$, $h = 20$, a random forest on $y_t, \dots, y_{t-4}$ and a time index; 60 replications; ratio of the estimated to the true RMSE on new data

How optimistic is each validation scheme?



- Ratio 1: the scheme estimates the error on new data without bias; below 1: optimistic

Interpreting the leakage experiment

- ▣ Random 5-fold: median ratio 0.34 (quartiles 0.28–0.39): it claims an error three times smaller than the truth
- ▣ Blocked 5-fold 0.92, purged 5-fold 0.98, last 20% 0.98: contiguity and the purge remove most of the bias
- ▣ The interior blocks still let the forest interpolate in time; only the last block reproduces extrapolation into the future
- ▣ Rule: purge at least h rows around every test block and drop time indices unless the trend is modelled explicitly

Recap: Learning from dependent data

- Generalisation bounds hold with an effective sample size set by mixing; blocking is the common tool
- Random K-fold is valid for autoregressions with uncorrelated errors, and then better than OOS
- Overlapping targets, under-fitting and time features break it: block, purge, and keep a final untouched period

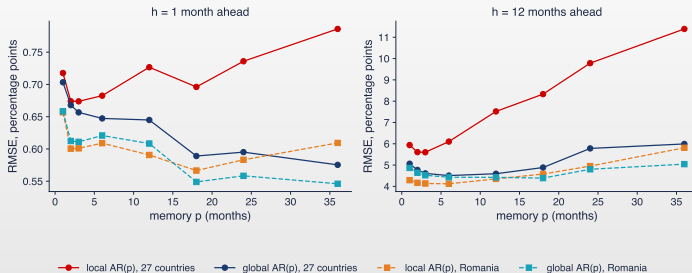
Local and global models

- **Local**: one model per series, $\hat{y}_{i,t+h} = f_i(y_{i,t}, y_{i,t-1}, \dots)$; **global**: one function for all series, $\hat{y}_{i,t+h} = f(y_{i,t}, y_{i,t-1}, \dots)$
 - ▶ global models pool $N \times T$ rows: far more data per parameter
- **Montero-Manso and Hyndman (2021)**: for any set of series a global model exists that forecasts as well as the local ones; global models can afford **longer memory** and more complexity
 - ▶ their bound: the complexity of a global model grows with NT , that of N local models with T each
- The M4 and M5 winners were global (**Smyl (2020)**; LightGBM in **Makridakis, Spiliotis and Assimakopoulos (2022)**): pooling, not depth, was the common ingredient
- Scaling matters: series of different levels are divided by their own scale before pooling (inflation rates share a scale)

Design: EU inflation, local against global

- ▣ Annual HICP inflation of the 27 EU countries; direct forecasts $h = 1$ and $h = 12$ months ahead from $AR(p)$ models, $p \in \{1, 2, 3, 6, 12, 18, 24, 36\}$
- ▣ Rolling window of 120 months; forecast origins from January 2015 (139 origins for $h = 1$)
- ▣ Local: 27 OLS regressions per origin; global: one pooled OLS (the same coefficients for all countries), as in [Montero-Manso and Hyndman \(2021\)](#)
- ▣ Loss: RMSE in percentage points; DM test on the cross-country average loss differential (HLN correction, [Chapter 1](#))

Memory helps global models and hurts local ones



□ RMSE against the memory p ; circles: average over the 27 countries; squares: Romania

Interpreting local and global

- ▣ $h = 1$: local RMSE is best at $p = 3$ (0.67) and then deteriorates; global keeps improving up to $p = 36$ (0.58); DM on the 27-country average -4.40 ($p < 0.001$)
- ▣ $h = 12$: the 2021–2023 surge dominates; local AR(36) reaches 11.39 against 5.99 global; best global 4.51 against best local 5.60, DM -1.09 (p 0.277)
- ▣ Romania alone: the global model helps at $h = 1$ with long memory (0.55 against 0.61 at $p = 36$), not significantly (DM p 0.19)
- ▣ Lesson of Montero-Manso and Hyndman confirmed: pooling buys memory; for a single series the gain is real but hard to prove

The lasso (1/2): optimality conditions

- Tibshirani (1996): least squares with an ℓ_1 penalty

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2n} \|y - X\beta\|^2 + \lambda \|\beta\|_1, \quad \|\beta\|_1 = \sum_j |\beta_j|$$

- ▶ y : $n \times 1$ target; X : $n \times p$ predictors, columns x_j ; $\lambda \geq 0$: penalty, the larger the more coefficients set to zero
- ▶ Karush–Kuhn–Tucker (KKT): $\frac{1}{n} x_j'(y - X\hat{\beta}) = \lambda \text{sign}(\hat{\beta}_j)$ if $\hat{\beta}_j \neq 0$, and $|\frac{1}{n} x_j'(y - X\hat{\beta})| \leq \lambda$ otherwise
- Orthonormal predictors ($X'X = nI$): soft thresholding of the OLS coefficient z_j

$$\hat{\beta}_j^{\text{lasso}} = \text{sign}(z_j) (|z_j| - \lambda)_+, \quad \hat{\beta}_j^{\text{ridge}} = \frac{z_j}{1 + \lambda}$$

- ▶ $(u)_+ = \max(u, 0)$; the lasso selects (sets small z_j to zero), ridge only shrinks

The lasso (2/2): prediction and selection

- Prediction: with s non-zero coefficients and a restricted eigenvalue κ , for $\lambda \asymp \sigma \sqrt{\log p/n}$

$$\frac{1}{n} \|X(\hat{\beta} - \beta)\|^2 = O_p\left(\frac{s \log p}{n \kappa^2}\right)$$

- ▶ σ : error standard deviation; κ : how far X is from collinearity on sparse directions; time series: the same rate for stable VARs [Basu and Michailidis \(2015\)](#), heavy-tailed errors [Medeiros and Mendes \(2016\)](#)
- Selection is harder than prediction: sign consistency needs the **irrepresentable condition** [Zhao and Yu \(2006\)](#)

$$\|X_{S^c}' X_S (X_S' X_S)^{-1} \text{sign}(\beta_S)\|_\infty < 1$$

- ▶ S : the set of relevant predictors, S^c : the others; it fails when relevant and irrelevant predictors are correlated (neighbouring countries, lags)

Adaptive lasso and elastic net

- ▣ **Adaptive lasso** Zou (2006): coefficient-specific weights from a first-step estimate

$$\lambda \sum_j w_j |\beta_j|, \quad w_j = 1/|\tilde{\beta}_j|^\gamma$$

- ▶ $\tilde{\beta}_j$: first-step estimate (OLS, ridge); $\gamma > 0$; large coefficients are penalised little, small ones a lot
- ▶ **oracle property**: consistent selection and $\sqrt{n}$ -normal estimates of the non-zero coefficients, without the irrerepresentable condition

- ▣ **Elastic net** Zou and Hastie (2005): a mix of the lasso and ridge penalties

$$\lambda \left(\alpha \|\beta\|_1 + \frac{1-\alpha}{2} \|\beta\|_2^2 \right)$$

$$\hat{\beta}_j = \frac{\text{sign}(z_j)(|z_j| - \lambda\alpha)_+}{1 + \lambda(1-\alpha)} \quad (X'X = nI)$$

- ▶ $\alpha \in [0, 1]$: $\alpha = 1$ lasso, $\alpha = 0$ ridge; grouping effect: correlated predictors enter together instead of one being chosen at random
- ▣ The penalty is tuned by CV: with h -step targets use blocked folds with a purge of h periods (Section 1)

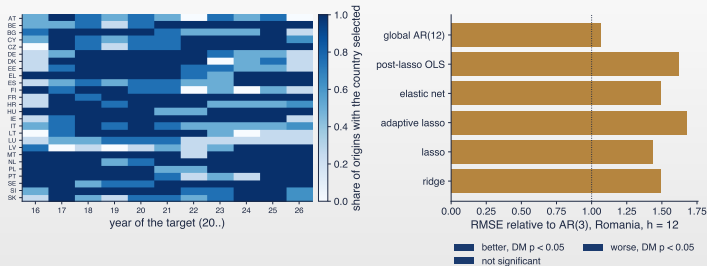
Inference after selection

- Naive practice: select with the lasso, then report OLS t -statistics of the selected variables: the p -values are **too small**
 - ▶ the same data chose the model and test it: the distribution of $\hat{\beta}$ is conditional on being selected
- Remedies: **PoSI** intervals valid for any selection [Berk et al. \(2013\)](#); **exact conditional** intervals for the lasso (a polyhedral selection event) [Lee et al. \(2016\)](#)
- For one coefficient of interest (a policy rate, a foreign shock): **double selection** [Belloni, Chernozhukov and Hansen \(2014\)](#): lasso of y on the controls, lasso of the regressor of interest on the controls, OLS on the union
 - ▶ with HAC standard errors for time series ([Chapter 0](#))
- For forecasting, selection is a means: judge it by out-of-sample loss and by the stability of what is selected

Design: Romanian inflation from the EU panel

- Target: Romanian annual HICP inflation 12 months ahead; window 120 months, re-estimation every 3 months; 128 forecasts (January 2016 – August 2026)
- Features: 3 own lags (never penalised) and 3 lags of the 26 other countries: 78 penalised predictors for about 105 observations
- Models: AR(3) (benchmark), ridge, lasso, adaptive lasso, elastic net ($\alpha = 0.5$), post-lasso OLS, global AR(12) of the panel; penalty by blocked 5-fold CV with a 12-month purge
- In the spirit of the data-rich inflation literature [Medeiros et al. \(2021\)](#), [Goulet Coulombe et al. \(2022\)](#), with the EU panel as the data set

What the panel lasso selects, and what it delivers



- Left: share of forecast origins in each year at which a country enters the lasso; right: RMSE relative to AR(3), colour by the DM test

Interpreting the panel lasso

- ▣ Selection is unstable: median 34 of 78 predictors (from 15 to 61); the most frequent are EL, NL, MT, not Romania's neighbours
- ▣ Accuracy: AR(3) RMSE 4.13; lasso 1.44, adaptive lasso 1.67, elastic net 1.49, ridge 1.49 times as large; global AR(12) 1.06
- ▣ None of the differences is significant (DM p from 0.20 to 0.39): about 10 independent 12-month periods, one dominated by the 2022 surge
- ▣ High dimension with 105 observations of a persistent target: CV picks small penalties, the panel fits past co-movement that does not repeat; parsimony wins here

Recap: Global models and high dimension

- Global models pool many series and can afford long memory; on EU inflation they win at one month
- The lasso predicts well under sparsity and restricted eigenvalues; it selects well only under the irrepresentable condition; adaptive lasso and elastic net relax it
- Report selection stability and use PoSI, exact or double-selection inference, never naive t -statistics

Bagging: the variance of an average

- Average of B predictors $\hat{f}_b(x)$ trained on bootstrap samples [Breiman \(1996\)](#)

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)\right) = \frac{1}{B^2} (B\sigma^2 + B(B-1)\rho\sigma^2) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- ▶ σ^2 : variance of one predictor; ρ : pairwise correlation of two predictors (over the data and the randomisation)
- ▶ more trees remove only the second term; the floor $\rho\sigma^2$ is lowered only by **decorrelating** the trees
- **Random forests** [Breiman \(2001\)](#): a random subset of m_{try} features at each split lowers ρ at the cost of a little bias
 - ▶ theory: consistency for additive regression [Scornet, Biau and Vert \(2015\)](#); honest trees (separate samples for splits and leaf values) give asymptotically Normal forecasts [Wager and Athey \(2018\)](#)
- Trees cannot extrapolate: a forecast is an average of past targets, so trends and level shifts need differencing or a linear part

Quantile regression forests

- [Meinshausen \(2006\)](#): keep **all** training targets in the leaves and weight them by how often they share a leaf with x

$$w_i(x) = \frac{1}{B} \sum_{b=1}^B \frac{1\{x_i \in L_b(x)\}}{|L_b(x)|}, \quad \hat{F}(y | x) = \sum_i w_i(x) 1\{y_i \leq y\}$$

- ▶ $L_b(x)$: the leaf of tree b that contains x ; $|L_b(x)|$: number of training points in it; $\sum_i w_i(x) = 1$
- ▶ $\hat{F}(y | x)$: estimated conditional distribution function; quantile $\hat{q}_\tau(x) = \inf\{y : \hat{F}(y | x) \geq \tau\}$
- The same forest gives every quantile: no crossing, no refit per level
 - ▶ consistent for the conditional distribution under regularity conditions; a random forest is an adaptive nearest-neighbour method
- Evaluation with the pinball loss and coverage ([Chapter 1](#)): the bands are honest only if the forest was trained on data like the future

Gradient boosting (1/2): functional gradient descent

- Friedman (2001): add small trees, each fitted to the negative gradient of the loss at the current fit

$$F_m = F_{m-1} + \nu h_m, \quad g_i = - \frac{\partial \ell(y_i, F)}{\partial F} \Big|_{F=F_{m-1}(x_i)}$$

- ▶ F_m : the fit after m trees; h_m : a small tree fitted to (x_i, g_i) ; $\nu \in (0, 1]$: learning rate (shrinkage): smaller steps, more trees, lower variance
- ▶ squared loss: $g_i = \text{residual}$; pinball loss: $g_i = \tau - 1\{y_i < F\}$ (quantile boosting)

Gradient boosting (2/2): second order, histograms, constraints

- ▣ Second order [Chen and Guestrin \(2016\)](#): optimal leaf value and the gain of a split

$$w^* = -\frac{G}{H + \lambda}, \quad \text{gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma$$

- ▶ G, H : sums of the first and second derivatives of the loss over the points of a leaf (L, R : left and right child); λ : penalty on leaf values; γ : cost of a split
- ▣ LightGBM [Ke et al. \(2017\)](#): features binned into histograms (at most 255 bins), leaf-wise growth, gradient-based one-side sampling; scikit-learn HistGradientBoosting (used here) follows the same construction
- ▣ Monotone constraints: a split is allowed only if the left and right leaf values respect the order, and children inherit bounds; the fit is then monotone in that feature everywhere

Design: Romanian day-ahead load



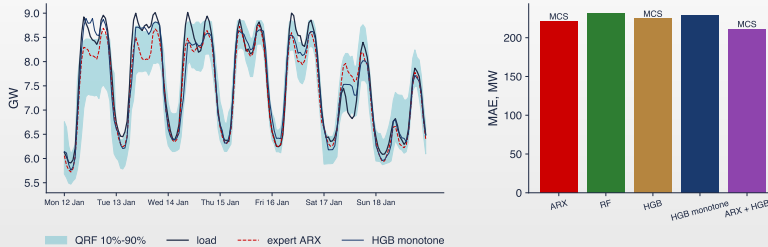
750 kV line at Mărașești, Romania

Photo: TrainSimFan (2022); CC BY-SA 4.0;

Wikimedia Commons

- Forecast the 24 hourly loads of day $d + 1$ with data up to day d ; evaluation 638 days, January 2025 – September 2026
- Benchmark: the expert ARX of [Chapter 1 \(Ziel and Weron \(2018\)\)](#): one regression per hour, 364-day window, refitted daily
- Global learners (one model for all hours) on the same lags, hour, weekday, Romanian holidays, annual cycle; refitted monthly on all earlier data: RF with QRF bands, HGB, HGB increasing in the three load lags, quantile HGB
- Losses: MAE, average pinball over the deciles, 80% coverage; DM against ARX and MCS ([Chapter 1](#))

Trees against the expert ARX

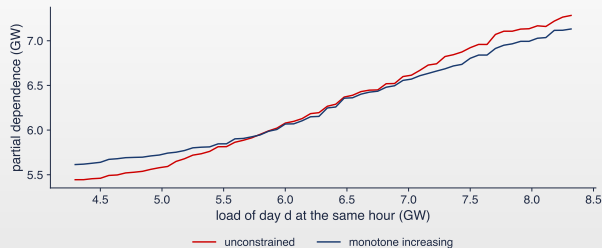


- Left: one January week with QRF 10%–90% bands; right: MAE in MW, MCS marks the models in the 90% Model Confidence Set

Interpreting the load comparison

- MAE (MW): ARX 221, RF 232, HGB 225, monotone HGB 228; the forest is significantly worse (DM 2.84), boosting is not significantly different from ARX
- The equal-weight average of ARX and boosting: 211 MW, DM -3.36 ($p < 0.001$), the only model in the MCS: combination beats selection ([Chapter 1](#))
- Quantiles: pinball (MW) ARX 89.1, quantile HGB 89.5, QRF 92.8; 80% coverage 78%, 61%, 83%: quantile boosting is sharp but under-covers, QRF over-covers
- A daily-refitted linear model with the right structure is a strong baseline; trees add value as a complement, not as a replacement

Monotone constraints in practice



- Partial dependence of the boosted forecast on the load of day d at the same hour, trained on 2023–2024

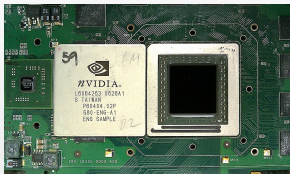
Interpreting the constraint

- Unconstrained: 3 decreasing steps (the largest 7.5 MW): more load yesterday would sometimes mean less load tomorrow, a fit to noise
- Constrained: monotone by construction; total range 1.52 GW against 1.84 GW: part of the effect moves to the other lags
- Accuracy barely changes (228 against 225 MW); the gain is plausibility and robustness at the edges of the data
- Constraints are how economic knowledge enters a black box: sign restrictions, as in [Chapter 3](#), but for nonlinear learners

Recap: Tree ensembles

- ▣ Bagging and forests reduce variance down to the floor $\rho\sigma^2$; decorrelation, not the number of trees, lowers it
- ▣ QRF gives the whole conditional distribution from one forest; boosting fits gradients, with second-order leaves and histograms in LightGBM
- ▣ On Romanian load, trees match but do not beat the expert ARX; their combination wins

From the MLP to sequence models



NVIDIA G80, the first CUDA graphics processor (2006)

Photo: Hyins (2011); public domain; Wikimedia Commons

- An MLP on a window of L lags is a nonlinear $\text{AR}(L)$

$$\hat{y}_{t+h} = W_2 \sigma(W_1 x_t + b_1) + b_2$$

- ▶ x_t : the L most recent values; W_1 , W_2 : weight matrices; b_1 , b_2 : biases; σ : activation applied elementwise (ReLU or tanh)

- Training: backpropagation [Rumelhart, Hinton and Williams \(1986\)](#) with Adam [Kingma and Ba \(2015\)](#); early stopping on the last block of the training sample; inputs standardised with training moments only
- Universal approximation says nothing about data needs: with a few thousand daily observations a network has little room over a linear model
- Sequence models share weights across time: recurrence (RNN), convolution (TCN) or attention (Transformer)

Recurrent networks and backpropagation through time (1/2)

- Elman RNN: a hidden state updated by the same weights at every step

$$h_t = \tanh(W h_{t-1} + U x_t + b), \quad \hat{y}_t = V h_t$$

- ▶ h_t : hidden state, a learned summary of the past (like the Kalman state of [Chapter 6](#)); x_t : input; W , U , V , b : weights shared over time

- Backpropagation through time (BPTT): the gradient sums contributions of all past steps

$$\frac{\partial \mathcal{L}_T}{\partial W} = \sum_{t \leq T} \frac{\partial \mathcal{L}_T}{\partial h_T} \left(\prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}} \right) \frac{\partial^+ h_t}{\partial W}$$
$$\frac{\partial h_k}{\partial h_{k-1}} = \text{diag}(1 - h_k^2) W$$

- ▶ $\mathcal{L}_T$: loss at time T ; $\partial^+ h_t / \partial W$: the direct effect of W at step t ; the contribution of lag $T - t$ is a product of $T - t$ Jacobians

Recurrent networks and backpropagation through time (2/2)

- Bengio, Simard and Frasconi (1994), Pascanu, Mikolov and Bengio (2013): the product of Jacobians is bounded geometrically

$$\left\| \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}} \right\| \leq (\gamma \|W\|)^{T-t}, \quad \gamma = \max |\tanh'| \leq 1$$

- ▶ $\gamma \|W\| < 1$: **vanishing** gradients, long lags cannot be learned
- ▶ spectral radius of W above 1: possible **explosion**, cured by gradient clipping

LSTM and GRU: gates (1/2)



Sepp Hochreiter, co-author of the
LSTM

Photo: Eulenreich (2015); CC BY-SA 4.0;
[Wikimedia Commons](#)

- LSTM Hochreiter and Schmidhuber (1997): three gates and a candidate, all from $[h_{t-1}, x_t]$

$$i_t, f_t, o_t = \sigma(W_{\bullet}[h_{t-1}, x_t] + b_{\bullet})$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

- ▶ i_t : input gate; f_t : forget gate; o_t : output gate; σ : logistic function, values in $(0, 1)$; $\tilde{c}_t$: candidate memory

- Cell and output

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t)$$

- ▶ $\odot$: elementwise product; along the cell $\partial c_t / \partial c_{t-1} = \text{diag}(f_t)$: additive, no squashing

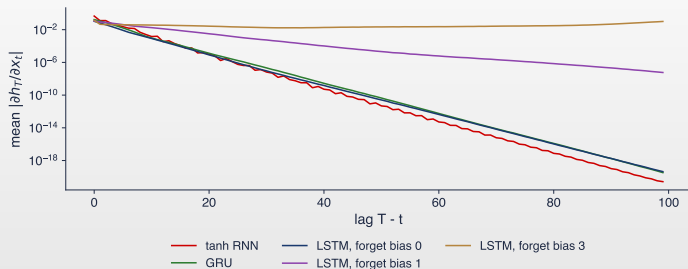
LSTM and GRU: gates (2/2)

- The forget gate [Gers, Schmidhuber and Cummins \(2000\)](#) must stay open: $f \approx 1$
 - ▶ a forget bias of 1 is the usual initialisation [Jozefowicz, Zaremba and Sutskever \(2015\)](#): $\sigma(1) = 0.73$ at the start of training
- GRU [Cho et al. \(2014\)](#): two gates, update z_t and reset r_t

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

- ▶ $\tilde{h}_t$: candidate state computed from $r_t \odot h_{t-1}$ and x_t ; fewer parameters than the LSTM, similar behaviour

How far back does the gradient reach?



□ Mean $|\partial h_T / \partial x_t|$ of freshly initialised networks (32 units, $T = 100$, Gaussian inputs, average of 5 seeds), log scale

Interpreting the gradients

- ▣ At lag 50 the gradient is a fraction 10^{-10} of its lag-1 value for the tanh RNN, 10^{-9} for the GRU and 10^{-9} for the LSTM with the default forget bias 0
- ▣ Forget bias 1: 10^{-4} ; forget bias 3: 0.47: gates help only when the forget gate is open ($\sigma(3) = 0.95$)
- ▣ Decay rates per lag: 0.51 (RNN), 0.45 (LSTM, bias 0), 0.03 (LSTM, bias 3): geometric in every case
- ▣ For daily series this matters less than it seems: HAR-type memory (22 lags) is within reach; for hourly data with weekly cycles (168 lags) it decides the architecture

Temporal convolutional networks

- Causal dilated convolution: a weighted sum of past inputs spaced d steps apart [van den Oord et al. \(2016\)](#)

$$z_t = \sum_{j=0}^{k-1} w_j x_{t-dj}$$

- ▶ k : kernel size; w_j : filter weights; d : dilation (skips $d - 1$ steps); only the past enters (causal)
- ▶ TCN [Bai, Kolter and Koltun \(2018\)](#): residual blocks of two dilated causal convolutions, $d = 1, 2, 4, \dots, 2^{\ell-1}$ over ℓ levels
- Receptive field: how far back the network can look

$$R = 1 + 2(k - 1)(2^{\ell} - 1)$$

- ▶ $k = 3, \ell = 4$: $R = 61$; one week of hours needs $\ell = 7$ ($R = 509$); memory is set by the architecture: no vanishing over time, but nothing beyond R
- Parallel over time (fast training), stable gradients; Bai et al. found TCNs competitive with LSTMs on standard sequence tasks

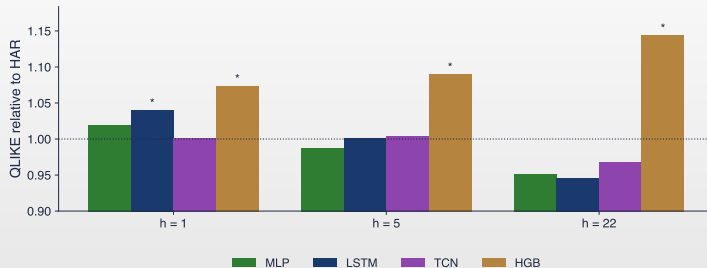
Multi-step forecasts and sequence-to-sequence

- ▣ **Recursive:** one-step model iterated, forecasts fed back as inputs: errors accumulate, the model is trained for $h = 1$ only
- ▣ **Direct:** one model per horizon h (our HAR and network forecasts); **MIMO:** one network outputs the whole vector $(\hat{y}_{t+1}, \dots, \hat{y}_{t+H})$ (DLinear, N-BEATS, Transformers here)
- ▣ **Encoder-decoder** [Sutskever, Vinyals and Le \(2014\)](#): an encoder RNN reads the past into a state, a decoder RNN unrolls the future; trained with teacher forcing
 - ▶ exposure bias: at test time the decoder sees its own errors; DeepAR samples paths instead
- ▣ Survey of recurrent forecasting practice [Hewamalage, Bergmeir and Bandara \(2021\)](#): deseasonalise or add seasonal lags, scale per series, train globally, ensemble seeds

Design: deep models against HAR

- Target $y = \log RV$ of the S&P 500 (Heber et al. (2009)); forecasts of RV_{t+h} , $h \in \{1, 5, 22\}$, direct; level forecast $\exp(\hat{m} + s^2/2)$ with the training residual variance
- Expanding window, first forecast after 2500 days (24 December 2009), re-estimation every 500 days; 3 051 forecasts per horizon
- Models on the last 22 values: HAR Corsi (2009) (OLS), MLP (2×32 units), LSTM (16 units), TCN (3 levels, $R = 29$), HGB on the 22 lags; one seed per network
- QLIKE Patton (2011), DM (HLN) against HAR, MCS; the literature: neural networks gain little over HAR Bucci (2020); ML gains appear mostly with many predictors Christensen, Siggaard and Veliyev (2023)

Can deep networks beat HAR?



□ QLIKE relative to HAR (1 = HAR); a star marks a DM rejection at 5%

Interpreting deep models against HAR

- ▣ $h = 1$: MLP 1.019, LSTM 1.040 (DM 2.77), TCN 1.002, HGB 1.074 (DM 3.84): nothing beats HAR, two are significantly worse
- ▣ $h = 22$: MLP 0.951, LSTM 0.946, TCN 0.968: gains of 3–5% that DM does not confirm (p 0.132, 0.444); HAR stays in the MCS (p 0.49)
- ▣ Boosting on raw lags is the worst at every horizon: trees cannot extrapolate to the volatility peaks of 2020
- ▣ HAR is a linear model with the right three features; a network must rediscover them from a few thousand days

Recap: Deep learning for sequences

- ▣ Gradients through time are products of Jacobians: they vanish geometrically unless the LSTM forget gate stays open
- ▣ TCNs fix the memory by the receptive field; multi-step forecasts are recursive, direct or MIMO
- ▣ On daily realised variance, networks do not significantly beat HAR at any horizon

Scaled dot-product attention

- Vaswani et al. (2017): each token attends to all tokens, with weights given by query–key similarity

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK'}{\sqrt{d_k}}\right)V$$

- ▶ $X \in \mathbb{R}^{n \times d}$: n tokens of dimension d ; W_Q, W_K, W_V : learned projections to queries, keys and values; d_k : key dimension; softmax normalises each row to weights that sum to 1
- ▶ each output is a weighted average of the values, with data-dependent weights; $\sqrt{d_k}$ keeps the logits of order one; multi-head: several projections in parallel, concatenated
- Permutation equivariant: without **positional encodings** the order of the tokens is lost, which is fatal for time series
- Cost $O(n^2d)$ in time and memory: one token per hour and a week of input give $n = 168$; a month gives $n = 720$
 - ▶ the long-horizon Transformers of 2020–2022 were mostly about this cost

Transformers for long horizons

- **Informer** Zhou et al. (2021): ProbSparse attention (only the most informative queries), self-attention distilling, a generative decoder that outputs the horizon at once: $O(n \log n)$
- **Autoformer** Wu et al. (2021): series decomposition inside every layer (moving-average trend and remainder) and an auto-correlation mechanism that aggregates sub-series at lags found by FFT
- **PatchTST** Nie et al. (2023): tokens are **patches** of consecutive values (e.g. 16 or 24), each channel modelled separately, instance normalisation
 - ▶ fewer tokens (n/P), local semantics in each token, longer look-backs affordable
- All were evaluated on the same handful of benchmarks (electricity, traffic, weather, ETT) with MSE on standardised data

Are Transformers effective? The Zeng et al. (2023) critique

- Zeng et al. (2023): self-attention is permutation-invariant; point-wise tokens carry no local semantics; positional encodings only partly restore order
- LTSF-Linear baselines: one linear map from the last L values to the next H

$$\textbf{Linear: } \hat{y}_{t+1:t+H} = W x_{t-L+1:t}$$

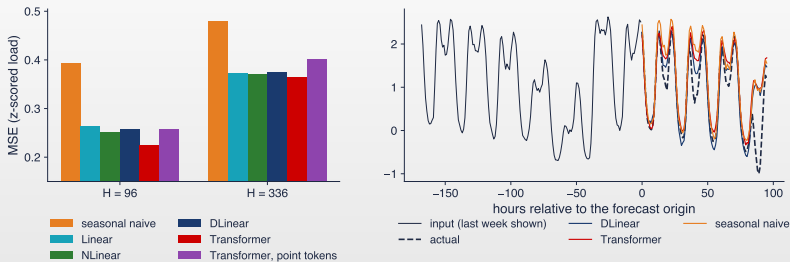
$$\textbf{DLinear: } \hat{y} = (W_s(I - A) + W_t A) x$$

- ▶ W : an $H \times L$ weight matrix; **NLinear** subtracts and adds back the last value; **DLinear**: A the moving-average matrix, W_t for the trend Ax , W_s for the remainder (Seminar 12, A8)
- Their finding: these one-layer models beat Informer, Autoformer and their successors on most of the standard long-horizon benchmarks
 - ▶ and the Transformers did not improve with a longer look-back L : a sign they did not use the extra history
- PatchTST was the reply: patching and channel independence put Transformers back ahead of DLinear on the same benchmarks

Replication design: the Zeng et al. protocol on Romanian load

- Hourly Romanian load as one series ($n = 32\,856$ hours), z-scored with training moments; chronological split 7:1:2; test from 31 December 2025
- Look-back $L = 336$ hours; horizons $H = 96$ and 336 ; MSE and MAE on the z-scored series, as in their tables
- Models: repeat the last value, seasonal naive (last week), Linear, NLinear, DLinear (kernel 25), a patch Transformer (14 patches of 24 hours, 1 layer, 4 heads) and the same Transformer with one token per hour
- Adam, MSE, early stopping on the validation block; three seeds averaged (one for the point-token Transformer, whose cost is $O(n^2)$)

Linear models, Transformers and the role of tokens



- Left: test MSE by horizon (repeating the last value, 1.585 and 1.699, is off the scale); right: one test window at $H = 96$ (the last week of input, the actual load and three forecasts)

Interpreting the protocol

- $H = 96$: seasonal naive 0.393, Linear 0.265, NLinear 0.253, DLinear 0.257; point-token Transformer 0.258; patch Transformer 0.226
- $H = 336$: DLinear 0.374, point tokens 0.402 (worse than every linear model), patches 0.364
- Both papers replicate on our data: the point-token Transformer is no better than NLinear or DLinear at $H = 96$ and worse than every linear model at $H = 336$; patching overturns the ranking
- The gap between the best linear model and the patch Transformer shrinks from 0.253 against 0.226 at $H = 96$ to 0.371 against 0.364 at $H = 336$: at long horizons the series is mostly seasonal level

The Temporal Fusion Transformer

- Lim et al. (2021): built for multi-horizon forecasts with three kinds of inputs: static (the store, the country), known future (calendar, prices), observed past
- Components
 - ▶ gated residual networks (GRN): $\text{LayerNorm}(a + \text{GLU}(\eta))$, a gate that can switch a block off
 - ▶ variable selection networks: softmax weights over the inputs at each time
 - ▶ static covariate encoders that condition every other block; an LSTM encoder–decoder for local patterns
 - ▶ interpretable multi-head attention (values shared across heads) for long-range dependence; quantile outputs trained with the pinball loss
- Its “interpretability” is the variable-selection weights and the attention pattern: useful diagnostics, not causal explanations (Section 7)

Recap: Attention and Transformers

- Attention is a data-dependent weighted average; order comes only from positional encodings; cost is quadratic in the tokens
- Zeng et al.: one linear layer beat point-token Transformers; PatchTST: patches restore the advantage; both replicate on Romanian load
- The TFT organises static, known and observed inputs with gates, selection and attention, and outputs quantiles

N-BEATS: basis expansion with residual stacking

- Oreshkin et al. (2020): each block reconstructs part of its input (backcast) and forecasts the rest

$$\hat{x}_\ell = g^b(\theta_\ell^b), \quad \hat{y}_\ell = g^f(\theta_\ell^f)$$
$$x_{\ell+1} = x_\ell - \hat{x}_\ell, \quad \hat{y} = \sum_\ell \hat{y}_\ell$$

- ▶ x_ℓ : input of block ℓ ; $\theta_\ell^b, \theta_\ell^f$: coefficients produced by four fully connected layers; g^b, g^f : basis functions; doubly residual: each block explains what the previous ones left
- **Generic**: g linear and learned; **interpretable**: trend basis $\{(t/H)^j\}_{j \leq 3}$ and seasonality basis $\{\cos 2\pi jt/H, \sin 2\pi jt/H\}$
 - ▶ H : forecast horizon; a learned decomposition, similar to exponential smoothing
- Pure deep learning, no time-series features; trained globally per frequency with sMAPE/MASE losses; the published results are ensembles of 180 networks
 - ▶ their M4 result beat the competition winner, the ES-RNN hybrid of [Smyl \(2020\)](#)

N-HiTS: multi-rate inputs, hierarchical outputs

- ▣ [Challu et al. \(2023\)](#): block ℓ first max-pools its input with kernel k_ℓ (e.g. 8, 4, 1): low-resolution blocks see the long-run shape, high-resolution blocks the detail
- ▣ Each block outputs only H/r_ℓ coefficients, interpolated to H points: the expressiveness ratio r_ℓ forces coarse blocks to forecast smooth components
 - ▶ an explicit frequency decomposition of the forecast, close in spirit to the wavelet multiresolution of [Chapter 11](#)
- ▣ Fewer parameters and less compute than N-BEATS for long horizons, with similar accuracy on short ones
- ▣ Ours: generic N-BEATS (3 blocks $\times$ 3 layers $\times$ 256 units) and N-HiTS (pools 8, 4, 1), input 336 hours, each window divided by its mean absolute level, MAE loss, median of 3 seeds

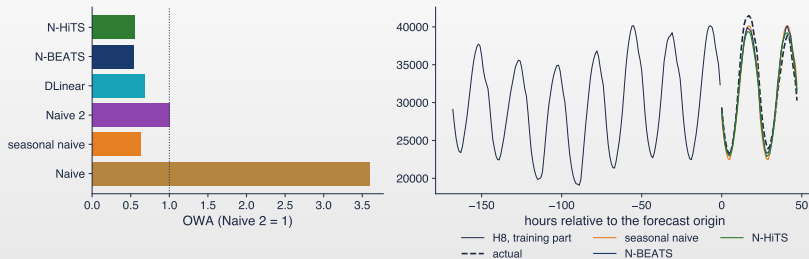
Design: the M4 hourly subset

- ▣ Makridakis, Spiliotis and Assimakopoulos (2020): 414 hourly series, horizon 48
- ▣ Accuracy measures: symmetric MAPE, scaled error and their average relative to Naive 2

$$\text{OWA} = \frac{1}{2} \left(\frac{\text{sMAPE}}{\text{sMAPE}_{\text{Naive2}}} + \frac{\text{MASE}}{\text{MASE}_{\text{Naive2}}} \right)$$

- ▶ sMAPE: mean of $2|y - \hat{y}|/(|y| + |\hat{y}|)$ in %; MASE: mean absolute error divided by the in-sample MAE of the seasonal naive (period 24) Hyndman and Koehler (2006); $\text{OWA} < 1$: better than Naive 2
- ▣ Our Naive, seasonal naive and Naive 2 reproduce the official M4 evaluation file exactly: Naive 2 sMAPE 18.38 (official 18.38), MASE 2.40 (official 2.40)
- ▣ Reference points from the same file: the M4 MLP and RNN benchmarks (OWA 0.921 and 1.036), Theta 1.006, the winner Smyl (2020) 0.440, the runner-up Montero-Manso et al. 0.484; all global models are trained only on the training parts

Global deep models on the M4 hourly series



□ Left: OWA (Naive 2 = 1); right: one series with the seasonal naive, N-BEATS and N-HiTS forecasts

Interpreting the M4 hourly results

- OWA: seasonal naive 0.628, DLinear 0.683, N-BEATS 0.539 (sMAPE 10.73, MASE 1.19), N-HiTS 0.548
- A few minutes of CPU training put N-BEATS ahead of every M4 benchmark and of the M4 neural benchmarks (0.921, 1.036), behind the two best entries (0.440, 0.484)
- The M4 neural benchmarks were local (one network per series); ours are global: pooling 414 series is what makes the difference
- DLinear is weaker here: hourly M4 series need nonlinear interactions of level and season that one linear layer cannot express

DeepAR: probabilistic forecasts from a global RNN

- Salinas et al. (2020): an LSTM state per series drives the parameters of a likelihood

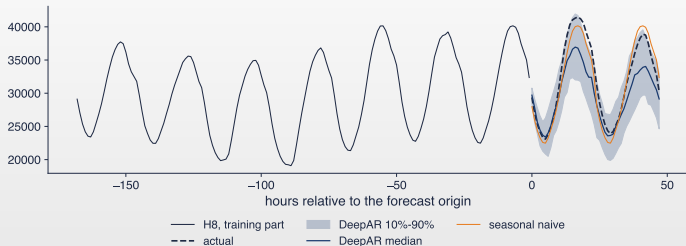
$$h_{i,t} = \text{LSTM}(h_{i,t-1}, z_{i,t-1}, x_{i,t}), \quad z_{i,t} \sim \ell(\cdot \mid \theta(h_{i,t}))$$

- ▶ $z_{i,t}$: value of series i at t ; $x_{i,t}$: covariates (lags at 1, 24, 168, calendar); $\theta(h)$: e.g. Gaussian (μ, σ) for real data, negative binomial for counts
- ▶ training: maximise $\sum_{i,t} \log \ell(z_{i,t} \mid \theta(h_{i,t}))$ over random windows of all series, each scaled by $1 +$ the mean of its context
- Forecast: ancestral sampling, each draw fed back as the next input; quantiles from the sample paths

$$\text{wQL} = \sum_{\tau} \frac{2 \sum_{i,t} \rho_{\tau}(z_{i,t} - \hat{q}_{\tau,i,t})}{\sum_{i,t} |z_{i,t}|}$$

- ▶ weighted quantile loss: ρ_{τ} the pinball loss at level τ , $\hat{q}_{\tau}$ the forecast quantile; an approximation of the CRPS Gneiting and Raftery (2007), lower is better
- The model of choice for large retail and energy panels; the base of several foundation models (Chapter 13)

A small DeepAR on the M4 hourly series



- Two-layer LSTM (40 units), context 168 hours, 4 000 training steps of 64 windows, 100 sample paths; the same series as on the previous chart

Interpreting DeepAR

- ▣ wQL by training budget (steps): 500: 0.0344, 1000: 0.0338, 2000: 0.0374, 4000: 0.0405; seasonal naive with empirical quantiles: 0.0375
- ▣ After 1000 steps DeepAR beats the benchmark (80% coverage 83%); with longer training it overfits the training windows and loses (76% at 4000 steps)
- ▣ Choosing the best checkpoint on this table would be data snooping: the budget must be fixed on a validation block before the test period is opened
- ▣ Probabilistic deep models are sensitive to the training budget; report the rule that set it, and the benchmark under a proper score

Lessons from the M5 competition

- ▣ [Makridakis, Spiliotis and Assimakopoulos \(2022\)](#): 42 840 Walmart series in a hierarchy, daily unit sales, 28 days ahead; accuracy and uncertainty tracks
 - ▶ the winning accuracy methods were global LightGBM models on engineered features (lags, rolling means, prices, calendar), often combined
- ▣ Global models, cross-learning and external information (prices, events) mattered more than architecture
- ▣ The simple benchmarks were beaten, but by smaller margins than ML marketing suggests, and least at the bottom level of the hierarchy
- ▣ An earlier warning: ML methods lost to statistical ones on the M3 monthly data [Makridakis, Spiliotis and Assimakopoulos \(2018\)](#); the difference between M3 and M5 is data size and cross-series learning

Recap: Global deep models

- ▣ N-BEATS and N-HiTS: residual stacks of fully connected blocks, generic or with trend and seasonal bases, multi-rate in N-HiTS
- ▣ On M4 hourly, global N-BEATS beats every benchmark and the local neural networks of M4
- ▣ DeepAR gives sample paths from a likelihood; it beats the seasonal naive under the wQL only at the right training budget, which must be set on validation data

Shapley values (1/2): the definition

- **Shapley (1953)**: the fair share of player g in a cooperative game v with G players

$$\phi_g = \sum_{S \subseteq \{1..G\} \setminus g} \frac{|S|! (G - |S| - 1)!}{G!} [v(S \cup g) - v(S)]$$

- ▶ $v(S)$: the value of coalition S ; $v(S \cup g) - v(S)$: the marginal contribution of g ; the weights average it over all orders in which players can join
- The unique allocation with four axioms
 - ▶ efficiency ($\sum_g \phi_g = v(\text{all}) - v(\emptyset)$), symmetry, dummy (a player who adds nothing gets 0) and additivity

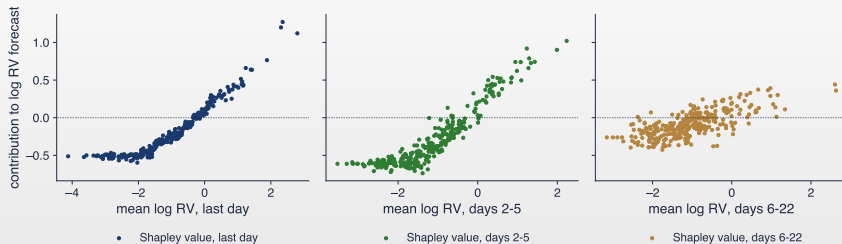
Shapley values (2/2): SHAP for forecasts

- SHAP [Lundberg and Lee \(2017\)](#): players are features; the value of a coalition S is the expected forecast when only x_S is known

$$v(S) = \mathbb{E}[f(x_S, X_{\bar{S}})]$$

- ▶ f : the fitted model; $\bar{S}$: the other features; **interventional**: $X_{\bar{S}}$ from a background sample, independent of x_S ; **observational**: conditional on x_S
- ▶ with lags, features are strongly correlated: interventional values evaluate the model at impossible histories; group correlated lags instead
- Here: exact Shapley values of three lag groups (day, days 2–5, days 6–22) for boosting on S&P 500 log RV, 300 forecast days, 100 background days

What the boosted model has learned



□ Each point: one forecast day; horizontal: the mean log RV of the group; vertical: its Shapley value

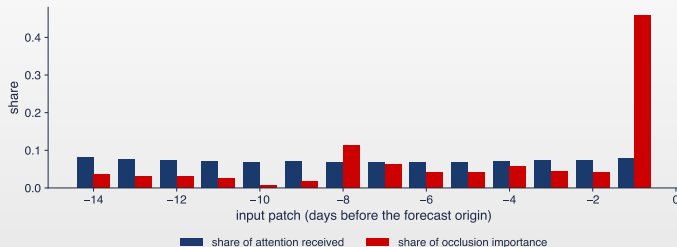
Interpreting the Shapley values

- Mean $|\phi|$ shares: last day 37%, days 2–5 46%, days 6–22 17%
- HAR on the same sample: coefficients 0.28, 0.50, 0.17 on the daily, weekly and monthly means: the boosted trees rediscovered the HAR cascade
- The values flatten at the extremes: trees cap their forecasts at the training range, the reason boosting lost to HAR in 2020
- Shapley values describe the model, not the data-generating process: a correct description of a wrong model is still wrong

Attention is not explanation

- Jain and Wallace (2019): attention weights are often uncorrelated with gradient-based importance, and very different attention patterns give the same predictions
- Wiegrefe and Pinter (2019): attention can be a faithful explanation under conditions, but this must be tested, not assumed
- A test: occlusion importance, the increase of the MSE when one input patch is replaced by the window mean, against the attention each patch receives
 - ▶ on the patch Transformer of Section 5 ($H = 96$), the first 1000 test windows

Where the Transformer looks, and what it uses



- Share of attention received by each of the 14 daily patches against the share of occlusion importance

 [ATS_ch12_interpretation](#)

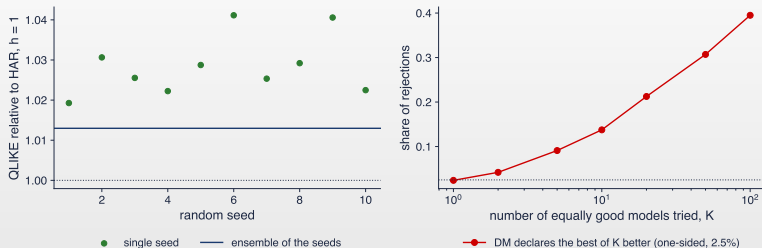
Interpreting attention and occlusion

- ▣ The last day carries 46% of the occlusion importance but receives only 8% of the attention
- ▣ Most attention goes to the patch 14 days back; the most important patch is 1 day back; Spearman correlation -0.09
- ▣ The flatten head reads the token states directly: what matters can bypass the attention weights entirely
- ▣ Report attention maps as what they are, a mixing pattern; for importance, use occlusion, Shapley or gradients, with a sanity check

The honest benchmark

- Strong baselines first: seasonal naive, ETS/Theta, HAR, an expert ARX; Lago et al. built an open benchmark because published electricity-price models were rarely compared with them [Lago et al. \(2021\)](#)
- Pitfalls catalogued by [Hewamalage, Ackermann and Bergmeir \(2023\)](#): test-set reuse, leakage in scaling and features, errors averaged across series of different scales, no significance tests, unreported tuning budget
- Tests ([Chapter 1](#)): DM for two models, MCS [Hansen, Lunde and Nason \(2011\)](#) for many, the reality check [White \(2000\)](#) and SPA [Hansen \(2005\)](#) when the best of many was chosen on the same data
- Report seeds, hardware, training time, every architecture tried; pre-register the design ([Chapter 0](#))

Seeds and data snooping



- Left: MLP against HAR for S&P 500 RV, $h=1$, with 10 seeds and their ensemble; right: simulation, K equally good models, 2500 test days, correlation 0.5

Interpreting seeds and snooping

- Seeds alone move the QLIKE ratio from 1.019 to 1.041 (median 1.027); the ensemble of the seeds reaches 1.013, better than every single seed
- Picking the best of K equally good models on the test set: the DM test declares it better (one-sided, nominal 2.5%) in 2% of cases for $K = 1$, 14% for $K = 10$, 40% for $K = 100$
- Hyper-parameter searches, seeds and architectures are all K : count them, and test the winner with SPA or MCS on data not used for the choice
- Ensembles are the cheap cure for seed variance; pre-registration is the cure for snooping

Recap: Interpretation and honest benchmarks

- ▣ Shapley values have axioms; with lagged features use groups and remember they describe the model
- ▣ Attention weights are not importance: on our Transformer they point the other way
- ▣ Strong baselines, DM/MCS/SPA, all seeds and all tries reported: otherwise any model can be made to win

An open question

- Do machine-learning and deep models forecast realised volatility better than HAR, across markets and horizons, once the comparison is pre-registered?
 - ▶ formal: $H_0: \mathbb{E}[\text{QLIKE}_{\text{ML}} - \text{QLIKE}_{\text{HAR}}] \geq 0$ in each cell; a cell supports ML only if DM rejects at 5% and the MCS excludes HAR
 - ▶ falsified if fewer than a fifth of the cells support ML
- Why it matters: risk systems (VaR, margins, option hedging) run on volatility forecasts; a spurious ML gain becomes a model risk
 - ▶ literature to start from: Corsi (2009), Bucci (2020), Christensen, Siggaard and Veliyev (2023), Hewamalage, Ackermann and Bergmeir (2023)

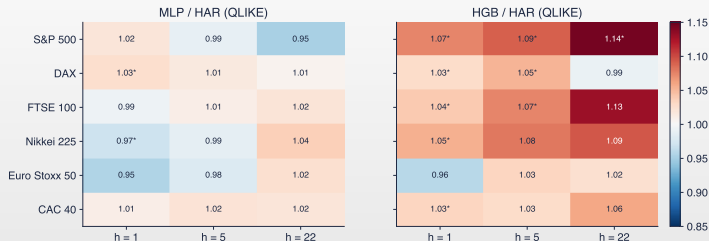
The discovery loop with an AI assistant

- An AI assistant (an LLM such as Claude, ChatGPT, Gemini or Copilot) speeds up each step; Semantic Scholar and Elicit help with the literature
 - ▶ **literature:** List peer-reviewed papers since 2019 comparing neural networks or boosting with HAR for realised volatility out of sample; give DOIs and the test used. Then check every DOI on Crossref
 - ▶ **design:** Write a pre-registration: assets, horizons, windows, refit schedule, loss, tests, seeds, what counts as a win.
 - ▶ **code and replication:** reproduce first the S&P 500 HAR QLIKE (0.253 at $h = 1$ here), then add learners one at a time
 - ▶ **critique:** Act as a hostile referee: list every source of leakage, snooping and unfair tuning in this comparison.
- Report: what was asked, what was kept, what was rejected (AI_USE.md, AI_ERRORS.md)

What the human checks

- ▣ Every reference exists and says what is claimed (DOI resolves, title matches, the result is in the paper)
- ▣ No future information: scaling, feature construction and tuning use only data before each origin; purge around h -step targets
- ▣ The baseline is tuned as carefully as the learner, and refitted on the same schedule
- ▣ Seeds, architectures and hyper-parameters tried are all reported; the winner is tested with MCS or SPA
- ▣ An AI claim that “LSTMs capture long memory, so they beat HAR” is checked against the numbers, not accepted

Mini-case: do learners beat HAR across markets?



- 36 cells: six indices $\times$ three horizons $\times$ two learners (MLP, boosting), the expanding design of Section 4; QLIKE ratio to HAR, a star where DM rejects at 5%
- Ratios from 0.95 to 1.14, median 1.024 (MLP 1.009, boosting 1.052); 9 cells below 1, 1 significantly better, 10 significantly worse: the hypothesis is falsified for these learners

Project idea

- ▣ **Deep learning against HAR for Central and Eastern European volatility:** replicate first, then extend
 - ▶ replicate: [Corsi \(2009\)](#) HAR and the neural comparison of [Bucci \(2020\)](#) on the Oxford-Man indices, with the design of Section 4
 - ▶ extend: intraday-based RV for the BET, WIG20 and EUR/RON; global models across markets; exogenous predictors (VIX, macro news) as in [Christensen, Siggaard and Veliyev \(2023\)](#)
 - ▶ pre-register: markets, horizons, windows, every learner and its tuning budget, QLIKE, DM, MCS, seeds
- ▣ Deliverables follow the course rules: repository, report, AI_USE.md, AI_ERRORS.md, oral defence

Key takeaways

- ▣ Validation must respect dependence: K-fold only for well-specified autoregressions; otherwise block, purge and keep a final test period
- ▣ Global models pool series and afford memory; high-dimensional selection on short macro samples is unstable
- ▣ Trees and networks rarely beat strong structured baselines alone (expert ARX, HAR); they help in combinations and in large panels
- ▣ Architecture details matter: open forget gates, patches instead of point tokens, global training
- ▣ Attention is not explanation; seeds and searches are hidden multiple testing

Self-assessment

Questions

- Why is random K-fold valid for an AR(3) fitted to AR(3) data but not for 22-day targets?
- Which condition does the adaptive lasso remove, and how?
- Why does adding trees not reduce the variance of a forest below $\rho\sigma^2$?
- What is the receptive field of a TCN with $k = 3$ and five levels?
- Why can a patch Transformer beat DLinear when a point-token Transformer cannot?

Next:

- Foundation models and conformal prediction
- pretrained models used zero-shot, and distribution-free intervals for any forecaster, including the ones of this chapter

Appendix: the bias of K-fold under autocorrelated errors

- Linear model $y = X\beta + \varepsilon$; leave out row i : $e_{(i)} = e_i / (1 - h_{ii})$, h_{ii} the leverage
- $\mathbb{E} e_{(i)}^2 \approx \sigma^2(1 + h_{ii}) - 2 \sum_{j \neq i} \frac{H_{ij}}{1 - h_{ii}} \text{Cov}(\varepsilon_i, \varepsilon_j)$ to first order
- Uncorrelated errors: the second term vanishes and LOOCV estimates the out-of-sample MSE $\sigma^2(1 + h)$: the Bergmeir–Hyndman–Koo case
- Positive correlation between the left-out error and the neighbouring training errors ($H_{ij} > 0$ for neighbours with similar lags): the estimate is too small; purging sets the neighbouring terms to zero

Appendix: soft thresholding and the elastic net

- With $X'X = nI$ the objective separates: $\frac{1}{2}(\beta_j - z_j)^2 + \lambda|\beta_j|$ for each j , $z_j = x_j'y/n$
- Subgradient: $0 \in \beta_j - z_j + \lambda \partial|\beta_j|$; if $|z_j| \leq \lambda$, $\beta_j = 0$; otherwise $\beta_j = z_j - \lambda \text{sign}(z_j)$
- Elastic net: $\frac{1}{2}(\beta_j - z_j)^2 + \lambda\alpha|\beta_j| + \frac{\lambda(1-\alpha)}{2}\beta_j^2$ gives $\beta_j = S(z_j, \lambda\alpha)/(1 + \lambda(1 - \alpha))$
- Adaptive lasso: threshold $\lambda w_j = \lambda/|\tilde{\beta}_j|^\gamma$, small for large $|\tilde{\beta}_j|$: the bias on large coefficients vanishes, which gives the oracle property

References (1/6)

- Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. [arXiv:1803.01271](https://arxiv.org/abs/1803.01271). [arxiv.org](https://arxiv.org/abs/1803.01271)
- Basu, S., & Michailidis, G. (2015). Regularized estimation in sparse high-dimensional time series models. *The Annals of Statistics*, 43(4), 1535–1567. [doi:10.1214/15-AOS1315](https://doi.org/10.1214/15-AOS1315)
- Belloni, A., Chernozhukov, V., & Hansen, C. (2014). Inference on treatment effects after selection among high-dimensional controls. *The Review of Economic Studies*, 81(2), 608–650. [doi:10.1093/restud/rdt044](https://doi.org/10.1093/restud/rdt044)
- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166. [doi:10.1109/72.279181](https://doi.org/10.1109/72.279181)
- Benidis, K., Rangapuram, S. S., Flunkert, V., Wang, Y., et al. (2022). Deep learning for time series forecasting: Tutorial and literature survey. *ACM Computing Surveys*, 55(6), 1–36. [doi:10.1145/3533382](https://doi.org/10.1145/3533382)
- Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191, 192–213. [doi:10.1016/j.ins.2011.12.028](https://doi.org/10.1016/j.ins.2011.12.028)
- Bergmeir, C., Hyndman, R. J., & Koo, B. (2018). A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120, 70–83. [doi:10.1016/j.csda.2017.11.003](https://doi.org/10.1016/j.csda.2017.11.003)
- Berk, R., Brown, L., Buja, A., Zhang, K., & Zhao, L. (2013). Valid post-selection inference. *The Annals of Statistics*, 41(2), 802–837. [doi:10.1214/12-AOS1077](https://doi.org/10.1214/12-AOS1077)
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140. [doi:10.1007/BF00058655](https://doi.org/10.1007/BF00058655)
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. [doi:10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324)
- Bucci, A. (2020). Realized volatility forecasting with neural networks. *Journal of Financial Econometrics*, 18(3), 502–531. [doi:10.1093/jjfinec/nbaa008](https://doi.org/10.1093/jjfinec/nbaa008)
- Burman, P., Chow, E., & Nolan, D. (1994). A cross-validated method for dependent data. *Biometrika*, 81(2), 351–358. [doi:10.1093/biomet/81.2.351](https://doi.org/10.1093/biomet/81.2.351)
- Challu, C., Olivares, K. G., Oreshkin, B. N., Garza Ramirez, F., Mergenthaler Canseco, M., & Dubrawski, A. (2023). NHITS: Neural hierarchical interpolation for time series forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(6), 6989–6997. [doi:10.1609/aaai.v37i6.25854](https://doi.org/10.1609/aaai.v37i6.25854)
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. [doi:10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785)

References (2/6)

- ▣ Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of EMNLP 2014*, 1724–1734. doi:10.3115/v1/D14-1179
- ▣ Christensen, K., Siggaard, M., & Veliyev, B. (2023). A machine learning approach to volatility forecasting. *Journal of Financial Econometrics*, 21(5), 1680–1727. doi:10.1093/jjfinec/nbac020
- ▣ Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2), 174–196. doi:10.1093/jjfinec/nbp001
- ▣ Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263. doi:10.1080/07350015.1995.10524599
- ▣ Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. doi:10.1214/aos/1013203451
- ▣ Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10), 2451–2471. doi:10.1162/089976600300015015
- ▣ Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359–378. doi:10.1198/016214506000001437
- ▣ Goulet Coulombe, P., Leroux, M., Stevanovic, D., & Surprenant, S. (2022). How is machine learning useful for macroeconomic forecasting? *Journal of Applied Econometrics*, 37(5), 920–964. doi:10.1002/jae.2910
- ▣ Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5), 2223–2273. doi:10.1093/rfs/hhaa009
- ▣ Hansen, P. R. (2005). A test for superior predictive ability. *Journal of Business & Economic Statistics*, 23(4), 365–380. doi:10.1198/073500105000000063
- ▣ Hansen, P. R., Lunde, A., & Nason, J. M. (2011). The model confidence set. *Econometrica*, 79(2), 453–497. doi:10.3982/ECTA5771
- ▣ Heber, G., Lunde, A., Shephard, N., & Sheppard, K. K. (2009). Oxford-Man Institute's realized library, version 0.3. Oxford-Man Institute, University of Oxford. web.archive.org
- ▣ Hewamalage, H., Ackermann, K., & Bergmeir, C. (2023). Forecast evaluation for data scientists: Common pitfalls and best practices. *Data Mining and Knowledge Discovery*, 37(2), 788–832. doi:10.1007/s10618-022-00894-5
- ▣ Hewamalage, H., Bergmeir, C., & Bandara, K. (2021). Recurrent neural networks for time series forecasting: Current status and future directions. *International Journal of Forecasting*, 37(1), 388–427. doi:10.1016/j.ijforecast.2020.06.008

References (3/6)

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. doi:10.1162/neco.1997.9.8.1735
- Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688. doi:10.1016/j.ijforecast.2006.03.001
- Jain, S., & Wallace, B. C. (2019). Attention is not explanation. *Proceedings of NAACL-HLT 2019*, 3543–3556. doi:10.18653/v1/N19-1357
- Jozefowicz, R., Zaremba, W., & Sutskever, I. (2015). An empirical exploration of recurrent network architectures. *Proceedings of the 32nd International Conference on Machine Learning*, PMLR 37, 2342–2350. proceedings.mlr.press
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30. proceedings.neurips.cc
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*. arXiv:1412.6980. arxiv.org
- Lago, J., Marcjasz, G., De Schutter, B., & Weron, R. (2021). Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark. *Applied Energy*, 293, 116983. doi:10.1016/j.apenergy.2021.116983
- Lee, J. D., Sun, D. L., Sun, Y., & Taylor, J. E. (2016). Exact post-selection inference, with application to the lasso. *The Annals of Statistics*, 44(3), 907–927. doi:10.1214/15-AOS1371
- Lim, B., Ank, S. Ö., Loeff, N., & Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4), 1748–1764. doi:10.1016/j.ijforecast.2021.03.012
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30. proceedings.neurips.cc
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4), 1346–1364. doi:10.1016/j.ijforecast.2021.11.013
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3), e0194889. doi:10.1371/journal.pone.0194889
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1), 54–74. doi:10.1016/j.ijforecast.2019.04.014
- Medeiros, M. C., & Mendes, E. F. (2016). ℓ_1 -regularization of high-dimensional time-series models with non-Gaussian and heteroskedastic errors. *Journal of Econometrics*, 191(1), 255–271. doi:10.1016/j.jeconom.2015.10.011

References (4/6)

- ▣ Medeiros, M. C., Vasconcelos, G. F. R., Veiga, Â., & Zilberman, E. (2021). Forecasting inflation in a data-rich environment: The benefits of machine learning methods. *Journal of Business & Economic Statistics*, 39(1), 98–119. doi:10.1080/07350015.2019.1637745
- ▣ Meinshausen, N. (2006). Quantile regression forests. *Journal of Machine Learning Research*, 7, 983–999. jmlr.org
- ▣ Mohri, M., & Rostamizadeh, A. (2010). Stability bounds for stationary φ -mixing and β -mixing processes. *Journal of Machine Learning Research*, 11, 789–814. jmlr.org
- ▣ Montero-Manso, P., & Hyndman, R. J. (2021). Principles and algorithms for forecasting groups of time series: Locality and globality. *International Journal of Forecasting*, 37(4), 1632–1653. doi:10.1016/j.ijforecast.2021.03.004
- ▣ Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). A time series is worth 64 words: Long-term forecasting with Transformers. *International Conference on Learning Representations*. arXiv:2211.14730. [arxiv.org](https://arxiv.org/abs/2211.14730)
- ▣ Oreshkin, B. N., Carpo, D., Chapados, N., & Bengio, Y. (2020). N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. *International Conference on Learning Representations*. arXiv:1905.10437. [arxiv.org](https://arxiv.org/abs/1905.10437)
- ▣ Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *Proceedings of the 30th International Conference on Machine Learning*, PMLR 28(3), 1310–1318. proceedings.mlr.press
- ▣ Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1), 246–256. doi:10.1016/j.jeconom.2010.03.034
- ▣ Petropoulos, F., Apiletti, D., Assimakopoulos, V., Babai, M. Z., et al. (2022). Forecasting: Theory and practice. *International Journal of Forecasting*, 38(3), 705–871. doi:10.1016/j.ijforecast.2021.11.001
- ▣ Racine, J. (2000). Consistent cross-validated model-selection for dependent data: hv-block cross-validation. *Journal of Econometrics*, 99(1), 39–61. doi:10.1016/S0304-4076(00)00030-0
- ▣ Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. doi:10.1038/323533a0
- ▣ Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3), 1181–1191. doi:10.1016/j.ijforecast.2019.07.001
- ▣ Scornet, E., Biau, G., & Vert, J.-P. (2015). Consistency of random forests. *The Annals of Statistics*, 43(4), 1716–1741. doi:10.1214/15-AOS1321
- ▣ Shapley, L. S. (1953). A value for n -person games. In H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the Theory of Games II*, 307–318. Princeton University Press. doi:10.1515/9781400881970-018

References (5/6)

- Smyl, S. (2020). A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *International Journal of Forecasting*, 36(1), 75–85. doi:10.1016/j.ijforecast.2019.03.017
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, 27. arXiv:1409.3215. [arxiv.org](https://arxiv.org/abs/1409.3215)
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288. doi:10.1111/j.2517-6161.1996.tb02080.x
- van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., & Kavukcuoglu, K. (2016). WaveNet: A generative model for raw audio. arXiv:1609.03499. [arxiv.org](https://arxiv.org/abs/1609.03499)
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. proceedings.neurips.cc
- Wager, S., & Athey, S. (2018). Estimation and inference of heterogeneous treatment effects using random forests. *Journal of the American Statistical Association*, 113(523), 1228–1242. doi:10.1080/01621459.2017.1319839
- White, H. (2000). A reality check for data snooping. *Econometrica*, 68(5), 1097–1126. doi:10.1111/1468-0262.00152
- Wiegreffe, S., & Pinter, Y. (2019). Attention is not not explanation. *Proceedings of EMNLP-IJCNLP 2019*, 11–20. doi:10.18653/v1/D19-1002
- Wu, H., Xu, J., Wang, J., & Long, M. (2021). Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in Neural Information Processing Systems*, 34. proceedings.neurips.cc
- Yu, B. (1994). Rates of convergence for empirical processes of stationary mixing sequences. *The Annals of Probability*, 22(1), 94–116. doi:10.1214/aop/1176988849
- Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), 11121–11128. doi:10.1609/aaai.v37i9.26317
- Zhao, P., & Yu, B. (2006). On model selection consistency of lasso. *Journal of Machine Learning Research*, 7, 2541–2563. jmlr.org
- Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(12), 11106–11115. doi:10.1609/aaai.v35i12.17325
- Ziel, F., & Weron, R. (2018). Day-ahead electricity price forecasting with high-dimensional structures: Univariate vs. multivariate modeling frameworks. *Energy Economics*, 70, 396–420. doi:10.1016/j.eneco.2017.12.016

References (6/6)

- Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2), 301–320. doi:10.1111/j.1467-9868.2005.00503.x
- Zou, H. (2006). The adaptive lasso and its oracle properties. *Journal of the American Statistical Association*, 101(476), 1418–1429. doi:10.1198/016214506000000735